

CXL Volatile Memory (VMEM) Emulation Report

1. Environment and Setup

The host/guest environment and QEMU/kernel build used for this track are produced by the same consolidated setup script used for the persistent-memory track, `CXL_Complete_Setup.sh`, rather than a separate manual build process.

Item	Value
Host OS	Ubuntu 24.04 LTS (native) or Ubuntu 24.04 LTS under WSL2, script auto-detects via <code>/proc/version</code>
Guest OS	Ubuntu 24.04.4 LTS (cloud image, cloud-init provisioned)
Guest Kernel	Linux v6.18, built with <code>CXL_BUS/PCI/ACPI/MEM/PORT/REGION</code> plus memory-hotplug and NUMA support
QEMU	Custom build from the jic23 CXL fork (branch <code>cxl-2025-03-20</code>)
CXL Tooling	<code>cxl-cli</code> (<code>cxl</code>), <code>ndctl</code> , <code>pciutils</code> , <code>stress-ng</code> , <code>numactl</code> , installed automatically if missing

The quick-start volatile launch script produced by this setup (`start-cxl.sh`) uses two Type 3 volatile devices on a single switch with a TCP QMP endpoint. The fuller dual-switch, four-device topology documented in this report (Sections 2–3) is built on the same QEMU binary and guest kernel, scaled up to two independent switches.

2. Objective

- Emulate CXL Type 3 volatile memory devices (System-RAM-type) across a dual-switch topology.
- Start the guest with a deliberately constrained 1 GiB RAM baseline and expose up to 16 GiB of addressable CXL capacity.
- Provision the CXL memory as regular System RAM (`cxl create-region -t ram`) rather than as persistent/pmem storage.
- Online the provisioned capacity via sysfs memory hotplug and confirm it is visible as a distinct NUMA node.
- Prove the extra capacity is real and usable with a 1.5 GiB over-allocation spillover stress test against the 1 GiB baseline.

3. Topology

The volatile track uses the same dual `cxl-rp` / `cxl-upstream` / `cxl-downstream` shape as the persistent-memory track, but each endpoint is backed by a volatile-memdev instead of a persistent-memdev, and no Label Storage Area (LSA) backing file is required:

```
pxb-cxl (bus_nr=12)
├── cxl-rp (port 0, chassis 0, slot 0) ← Switch A
│   ├── cxl-upstream (us0)
│   │   ├── cxl-downstream (ds0a) → cxl-type3 vmem0 (sn=0x10)
│   │   └── cxl-downstream (ds1a) → cxl-type3 vmem1 (sn=0x11)
└── cxl-rp (port 1, chassis 0, slot 1) ← Switch B
    └── cxl-upstream (us1)
```

```

└─ cxl-downstream (ds0b) → cxl-type3 vmem2 (sn=0x20)
└─ cxl-downstream (ds1b) → cxl-type3 vmem3 (sn=0x21)

```

Component	QEMU Device	Description
CXL Host Bridge	pxb-cxl	bus_nr=12, gateway between PCIe and CXL domains
CXL Root Ports	cxl-rp × 2	One per switch (Switch A, Switch B)
CXL Upstream Ports	cxl-upstream × 2	us0 (Switch A), us1 (Switch B)
CXL Downstream Ports	cxl-downstream × 4	Two per switch, one Type 3 device each
CXL Type 3 Devices	cxl-type3 × 4	vmem0–vmem3, volatile-memdev, 1 GiB each
Memory Backends	cxl-volatile-mem0..3.raw	4 × 1 GiB memory-backend-file
Fixed Memory Window	cxl-fmw.0	8 GiB, 4k interleave granularity
Base Guest RAM	-m 1G,maxmem=16G,slots=8	Deliberately constrained baseline for spillover testing

4. Emulation Script (emulation.sh)

2.1 Backing File Preparation

Four 1 GiB raw files are created to back the volatile memory devices. Unlike the persistent-memory track, no LSA files are needed since volatile devices carry no label metadata:

```

dd if=/dev/zero of=cxl-volatile-mem0.raw bs=1M count=1024
dd if=/dev/zero of=cxl-volatile-mem1.raw bs=1M count=1024
dd if=/dev/zero of=cxl-volatile-mem2.raw bs=1M count=1024
dd if=/dev/zero of=cxl-volatile-mem3.raw bs=1M count=1024
chmod 660 cxl-volatile-*.raw

```

2.2 QEMU Launch Command

QEMU is launched with KVM acceleration, a 1 GiB baseline (maxmem=16G), and the dual-switch volatile topology. Networking (SSH forward on 2222), a VirtFS host-share for report capture, and serial console output are also configured:

```

-machine q35,cxl=on,accel=kvm
-cpu host,migratable=off
-smp 4
-m 1G,maxmem=16G,slots=8
-kernel "${KERNEL}"
-append "root=/dev/vda1 rootwait rootdelay=5 console=ttyS0,115200 earlyprintk=ttyS0 rw"
-drive file="${GUEST_DISK}",format=qcow2,if=none,id=hd0
-device virtio-blk-pci,drive=hd0,bus=pcie.0
-netdev user,id=net0,hostfwd=tcp::2222-:22
-device virtio-net-pci,netdev=net0,bus=pcie.0
-virtfs local,path="${REPORTS_DIR}",mount_tag=hostshare,security_model=mapped-xattr,id=share
-nographic -d guest_errors

```

```

-object memory-backend-file,id=vmem0,share=on,mem-path="${IMAGES}/cxl-volatile-mem0.raw",size=1G
-object memory-backend-file,id=vmem1,share=on,mem-path="${IMAGES}/cxl-volatile-mem1.raw",size=1G
-object memory-backend-file,id=vmem2,share=on,mem-path="${IMAGES}/cxl-volatile-mem2.raw",size=1G
-object memory-backend-file,id=vmem3,share=on,mem-path="${IMAGES}/cxl-volatile-mem3.raw",size=1G

-device pxb-cxl,bus_nr=12,bus=pcie.0,id=cxl.1

-device cxl-rp,port=0,bus=cxl.1,id=rp0,chassis=0,slot=0
-device cxl-upstream,bus=rp0,id=us0
-device cxl-downstream,port=0,bus=us0,id=ds0a,chassis=1,slot=1
-device cxl-downstream,port=1,bus=us0,id=ds1a,chassis=1,slot=2
-device cxl-type3,bus=ds0a,volatile-memdev=vmem0,id=cxl-dev0,sn=0x10
-device cxl-type3,bus=ds1a,volatile-memdev=vmem1,id=cxl-dev1,sn=0x11

-device cxl-rp,port=1,bus=cxl.1,id=rp1,chassis=0,slot=1
-device cxl-upstream,bus=rp1,id=us1
-device cxl-downstream,port=0,bus=us1,id=ds0b,chassis=2,slot=1
-device cxl-downstream,port=1,bus=us1,id=ds1b,chassis=2,slot=2
-device cxl-type3,bus=ds0b,volatile-memdev=vmem2,id=cxl-dev2,sn=0x20
-device cxl-type3,bus=ds1b,volatile-memdev=vmem3,id=cxl-dev3,sn=0x21

-M cxl-fmw.0.targets.0=cxl.1,cxl-fmw.0.size=8G,cxl-fmw.0.interleave-granularity=4k

```

Note: unlike the persistent-memory launch script, `clflushopt/clwb` are not exposed or masked here, since volatile CXL memory is treated as ordinary System RAM and does not rely on cache-flush-to-persistence instructions.

5. Operations Script (operations.sh)

Once the guest is booted, a six-step in-guest workflow provisions the volatile CXL capacity as usable System RAM and proves it can absorb memory pressure beyond the 1 GiB baseline.

Step 1, Topology Verification

Confirms all four volatile memdevs and their decoders are visible before provisioning begins:

```

cxl list -BMPD
cxl list -M
cxl list -D

```

Step 2, Region Provisioning (1-way, RAM-type)

Any stale regions are torn down first. Each memdev is then provisioned as its own 1-way (non-interleaved), RAM-type region rather than combined into an interleaved region. This is a deliberate workaround: interleaved volatile regions were found to trigger a KVM MMIO limitation (Suberror 3) when the kernel zeroes newly onlined memory, so each device is kept on its own contiguous 1-way region instead:

```

sudo cxl disable-region all
sudo cxl destroy-region all

sudo cxl create-region -t ram -m mem0 -d decoder0.0 -w 1 -s 1G
sudo cxl create-region -t ram -m mem1 -d decoder0.0 -w 1 -s 1G
sudo cxl create-region -t ram -m mem2 -d decoder0.0 -w 1 -s 1G
sudo cxl create-region -t ram -m mem3 -d decoder0.0 -w 1 -s 1G

```

Step 3, Onlining Memory via sysfs

Every offline memory block exposed under `/sys/devices/system/memory/` is walked and switched online, folding the CXL capacity into the kernel's general-purpose System RAM pool:

```

for mem in /sys/devices/system/memory/memory*/state; do
    if grep -q "offline" "$mem" 2>/dev/null; then
        echo online | sudo tee "$mem" >/dev/null
    fi
done

```

`free -h` is captured both before and after this step to show the total available RAM increase.

Step 4, NUMA Topology Verification

`numactl --hardware` confirms the onlined CXL capacity is enumerated as a distinct NUMA node rather than being silently merged into node 0, matching expected CXL behavior for memory-expansion use cases. `stress-ng` and `numactl` are installed on demand if not already present.

Step 5, 1.5 GiB Over-Allocation Spillover Test

This is the core proof point of the volatile track. With the guest's base RAM fixed at 1 GiB, the kernel overcommit policy is temporarily relaxed and `stress-ng` is asked to allocate and touch 1.5 GiB, 500 MiB more than the baseline can hold on its own:

```

sudo sysctl -w vm.overcommit_memory=1

sudo stress-ng \
    --vm 1 \
    --vm-bytes 1500m \
    --timeout 20s \
    --metrics-brief

sudo sysctl -w vm.overcommit_memory=0

```

Test principle: if CXL volatile memory were not functioning as real, usable RAM, this step would OOM or crash the guest. A clean completion demonstrates that the extra 500 MiB was serviced from the onlined CXL capacity.

Step 6, Final State Summary

free -h and numactl --hardware are captured a final time to document the post-test memory state.

6. Results

Check	Result
Topology discovery	4 volatile memdevs (vmem0–vmem3) visible across both switches
Region provisioning	4 × 1 GiB RAM-type regions, 1-way interleave, all committed
Memory onlining	All offline blocks under /sys/devices/system/memory/ brought online
NUMA visibility	CXL capacity enumerated as a separate NUMA node from base RAM
Spillover test	stress-ng requested 1.5 GiB against a 1 GiB baseline and completed without OOM or crash
Conclusion	CXL-backed volatile memory functions as genuine, addressable System RAM under KVM

7. Known Limitations

- Interleaved volatile regions are not used across devices; each memdev is provisioned as its own 1-way region to avoid a KVM MMIO limitation (Suberror 3) triggered when the kernel zeroes newly onlined interleaved memory.
- CXL Machine Check Exception (MCE) reporting remains unsupported in QEMU's CXL emulation, expected and does not affect the spillover test outcome.
- The spillover test demonstrates functional capacity, not performance; no bandwidth/latency comparison between base RAM and CXL-backed capacity is included in this script.

8. Conclusion

The volatile CXL emulation track successfully models a dual-switch, four-device System-RAM expansion topology under KVM acceleration. Region provisioning, sysfs onlining, and NUMA enumeration all behave as expected, and the 1.5 GiB over-allocation spillover test confirms that CXL-backed volatile memory is usable as real, addressable RAM rather than merely being discoverable at the topology level.